DEVOPS CAPSTONE

CampusDesk

A campus IT helpdesk, delivered from laptop to GitOps-managed Kubernetes



T. Abdul Kalam Azad
24BCS10053

Campus IT requests need one clear queue

Report

Students and staff raise network, hardware, software or access issues

Prioritise

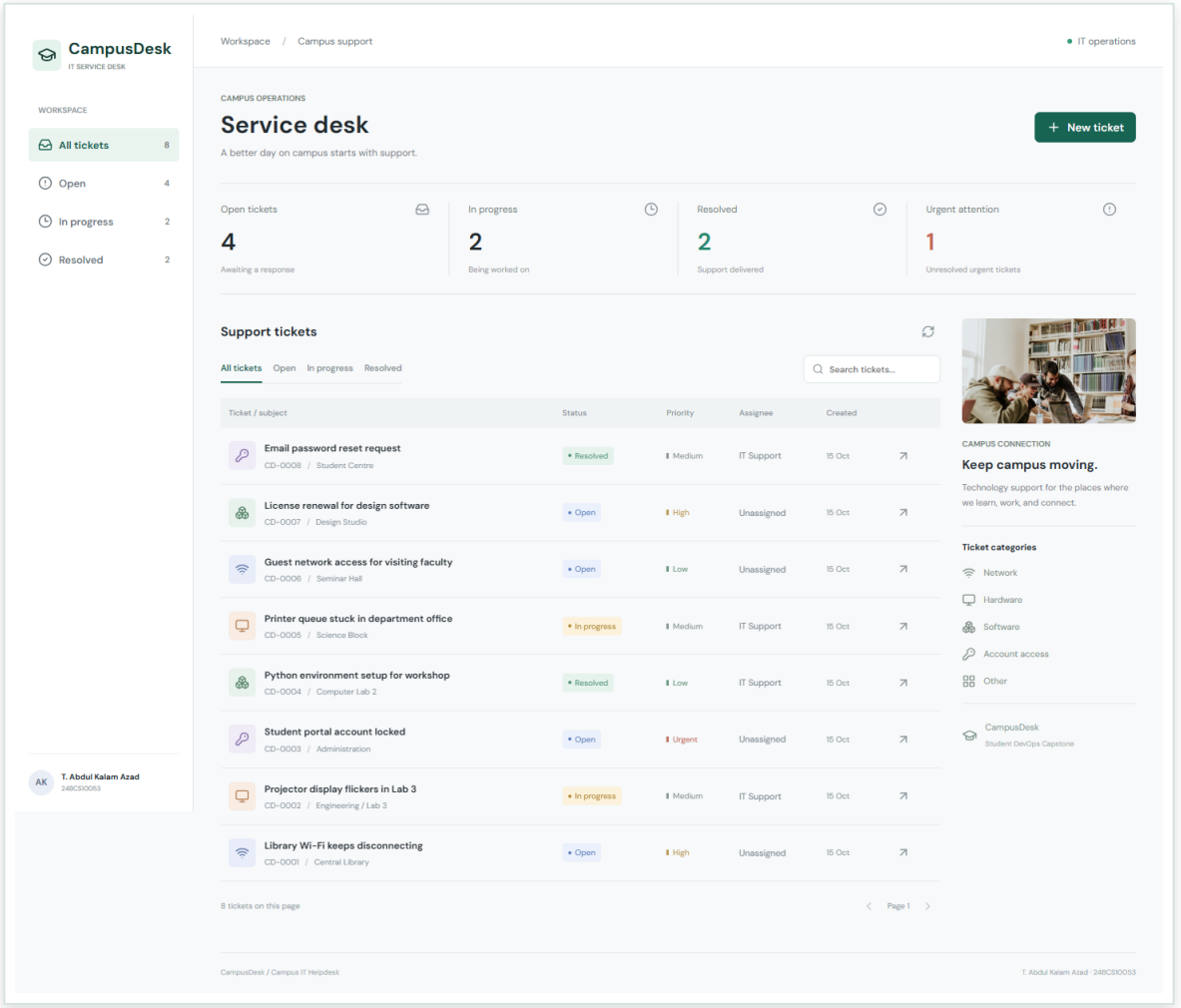
Support sets category, priority (low to urgent) and an assignee

Track

Status filters and search across title, requester and location

Resolve

Tickets move open, in progress, resolved; live stats on top



One commit travels the whole delivery path



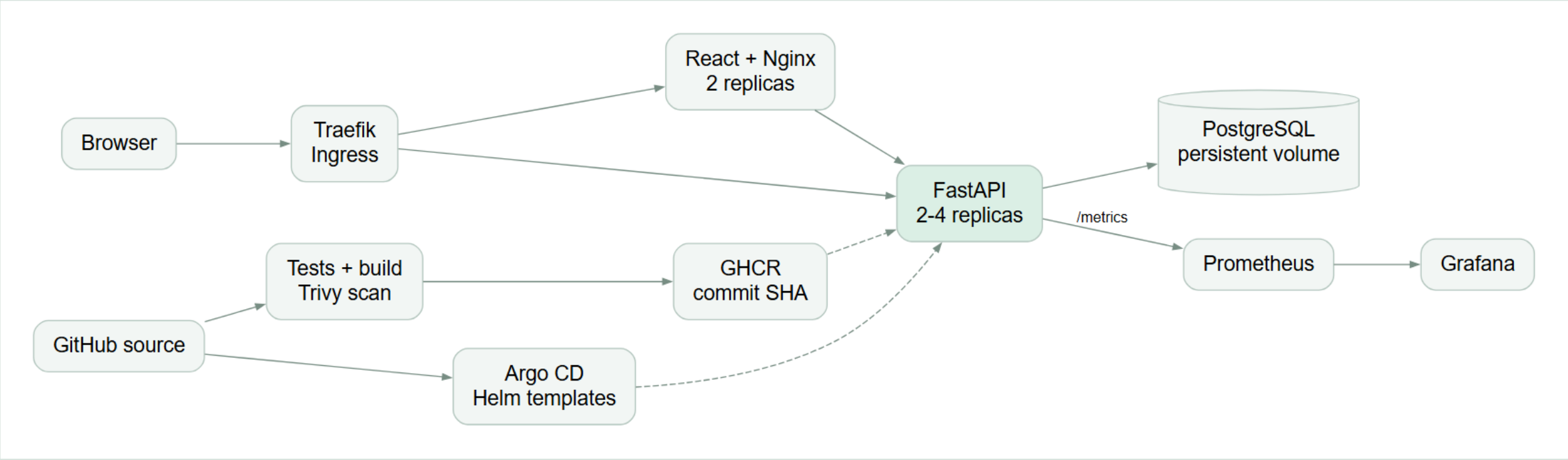
Terraform provisions the platform

VPC with 2 public + 2 private subnets, NAT gateway, EKS and a managed node group in ap-south-1. The same Helm chart deploys to it with values-aws.yaml.

Prometheus + Grafana watch it run

Every backend pod exposes /metrics. Prometheus scrapes each replica; Grafana shows traffic, P95 latency and errors, with two alert rules.

Architecture at a glance



Relative /api calls

The browser never knows a backend hostname. The Ingress sends /api to FastAPI and / to the React frontend.

Migrations run once

An Alembic Job (Argo CD sync hook) upgrades the schema before the backend becomes ready.

One chart, two clusters

The same Helm chart runs locally on kind and on Terraform-built EKS with values-aws.yaml.

Tech stack

Frontend

React 19, Vite 6, hand-written responsive CSS, served by unprivileged nginx

Backend

Python 3.13, FastAPI, SQLAlchemy, Pydantic, Uvicorn

Database

PostgreSQL 17 with Alembic migrations

Testing

pytest (SQLite locally, PostgreSQL in CI), Ruff, Playwright

Containers

Non-root Alpine backend, multi-stage frontend, Docker Compose

CI/CD + security

GitHub Actions, Trivy, Bandit, pip-audit, npm audit, Gitleaks, GHCR

Kubernetes

Helm, Traefik Ingress, HPA with metrics-server, Argo CD

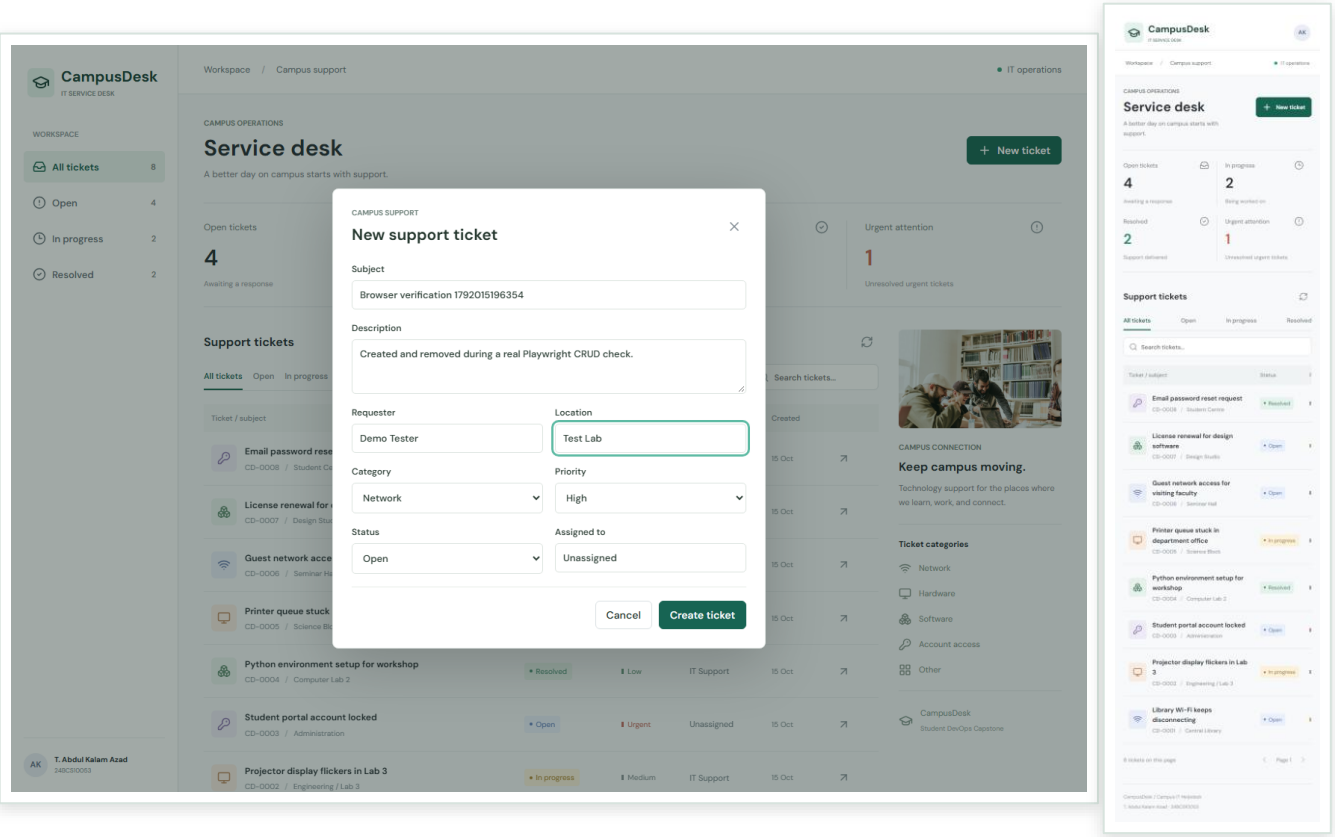
Infrastructure

Terraform: AWS VPC, NAT, EKS, managed node group, EBS CSI

Observability

Prometheus custom metrics, Grafana dashboard, alert rules

The application and its REST API



Method	Path	Purpose
GET	`/api/tickets`	list, filter, search, paginate
GET	`/api/tickets/{id}`	one ticket or 404
POST	`/api/tickets`	validate and create (201)
PUT	`/api/tickets/{id}`	update editable fields
DELETE	`/api/tickets/{id}`	delete (204)
GET	`/api/stats`	counts incl. urgent
GET	`/health` `/ready`	liveness, readiness
GET	`/metrics`	Prometheus format

Invalid input returns 422, failed writes roll back with 503, and search escapes SQL wildcards. Full docs live at /docs.

Tests are the first quality gate

40

pytest cases, all passing

Never the app database

Isolated SQLite locally, a dedicated PostgreSQL in CI; the suite refuses to start if both URLs match.

Real migration, rolled back

Schema built by Alembic; every test runs in a transaction that is rolled back.

Browser checks

Playwright drives real CRUD, validation and narrow screens against the kind deployment.

CampusDesk | Captured command output: pytest

Original recorded output, rendered for readability. Environment: local Windows / Kind.

```
Captured at 2026-10-14T22:06:49.976497+00:00
Command: C:\Users\abdul\Downloads\devops\scaler-devops-homework\capstone-campusdesk\.venv\Scripts\python.exe -m pytest -v
Exit code: 0

============================= test session starts =============================
platform win32 -- Python 3.13.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\abdul\Downloads\devops\scaler-devops-homework\capstone-campusdesk\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\abdul\Downloads\devops\scaler-devops-homework\capstone-campusdesk\backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collecting ... collected 20 items

tests/test_api.py::test_health_ready_metrics PASSED                      [  5%]
tests/test_api.py::test_crud PASSED                                      [ 10%]
tests/test_api.py::test_filters_and_stats PASSED                         [ 15%]
tests/test_api.py::test_validation[title- ] PASSED                       [ 20%]
tests/test_api.py::test_validation[requester-] PASSED                    [ 25%]
tests/test_api.py::test_validation[priority-critical] PASSED             [ 30%]
tests/test_api.py::test_validation[status-closed] PASSED                 [ 35%]
tests/test_api.py::test_validation[category-invalid] PASSED              [ 40%]
tests/test_api.py::test_validation[location-A] PASSED                    [ 45%]
tests/test_api.py::test_validation[unknown-extra] PASSED                 [ 50%]
tests/test_api.py::test_missing_ticket[get] PASSED                       [ 55%]
tests/test_api.py::test_missing_ticket[put] PASSED                       [ 60%]
tests/test_api.py::test_missing_ticket[delete] PASSED                    [ 65%]
tests/test_api.py::test_invalid_query PASSED                             [ 70%]
tests/test_api.py::test_search_escapes_wildcards PASSED                  [ 75%]
tests/test_api.py::test_readiness_failure PASSED                         [ 80%]
tests/test_api.py::test_unhandled_failure_is_counted PASSED              [ 85%]
tests/test_api.py::test_metrics_use_bounded_routes_and_exclude_scrapes PASSED [ 90%]
tests/test_api.py::test_invalid_update_preserves_existing_ticket PASSED  [ 95%]
tests/test_api.py::test_pagination_has_stable_order PASSED               [100%]

============================== warnings summary ===============================
..\.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\abdul\Downloads\devops\scaler-devops-homework\capstone-campusdesk\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using
`httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 20 passed, 1 warning in 0.57s ========================
```

Two small, non-root container images

`campusdesk-backend`

- python:3.13-alpine, patched with apk upgrade
- Dependencies installed before code for layer caching
- pip and setuptools removed after install
- Runs as uid 10001, HEALTHCHECK on /health

`campusdesk-frontend`

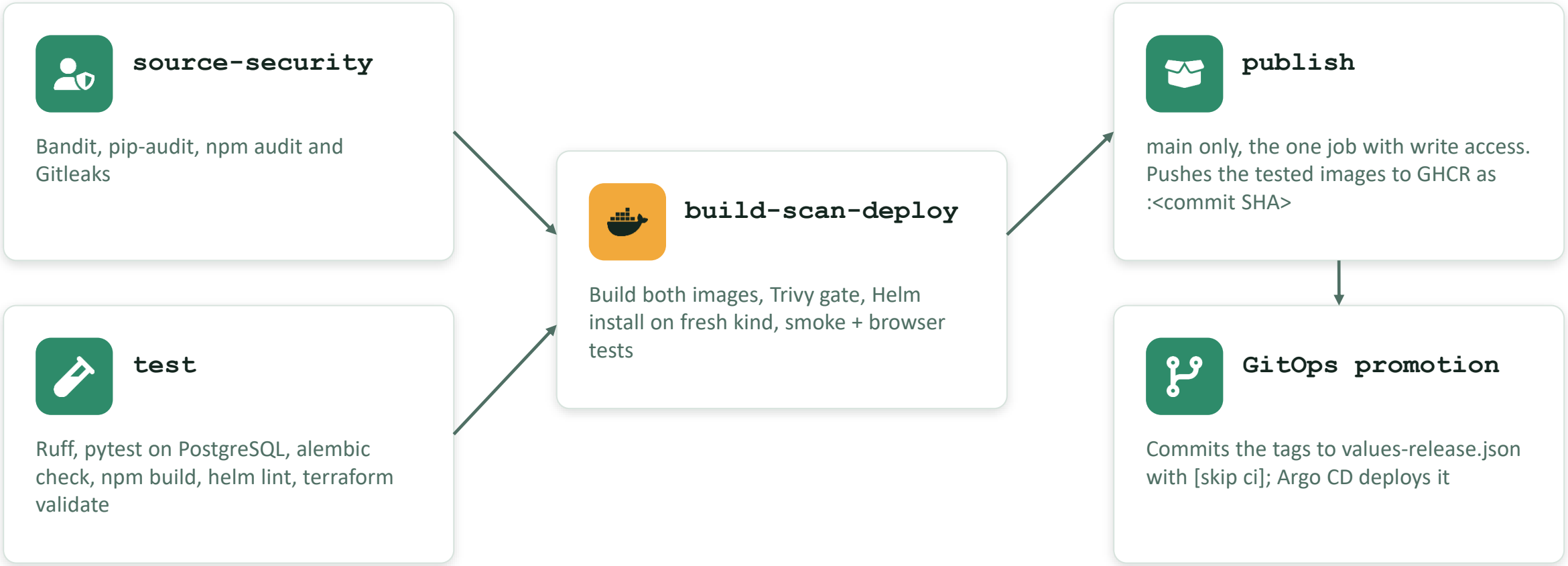
- Stage 1: node:24-alpine runs npm ci + build
- Stage 2: nginx-unprivileged serves only dist/
- Node never reaches the runtime image
- Runs as uid 101 on port 8080

`docker compose up --build`

- postgres
- migrate (one-shot)
- backend :8000
- frontend :3000

The backend image cannot install packages at runtime, which also removes the setuptools CVE path shown on the security slide.

CI/CD: every gate must pass before publish



Triggered on every push to main, on capstone pull requests, and manually. **No continue-on-error, no latest tag, main runs never cancelled.**

DevSecOps: a real CVE, found and fixed

48

findings on the first Debian-based backend image

→

0

HIGH/CRITICAL on both published images today

CVE-2025-47273 (HIGH)

A path-traversal flaw in old setuptools' package-index code: a crafted package name could write files outside the intended directory. Fix: rebuild on patched Alpine and remove pip/setuptools after installing dependencies, because the running service never installs packages.

A clean scan means no known HIGH/CRITICAL CVE at scan time, not proof the app is secure. That is why every build rescans.



SAST

Bandit



Dependencies

pip-audit, npm audit



Secrets

Gitleaks



Containers

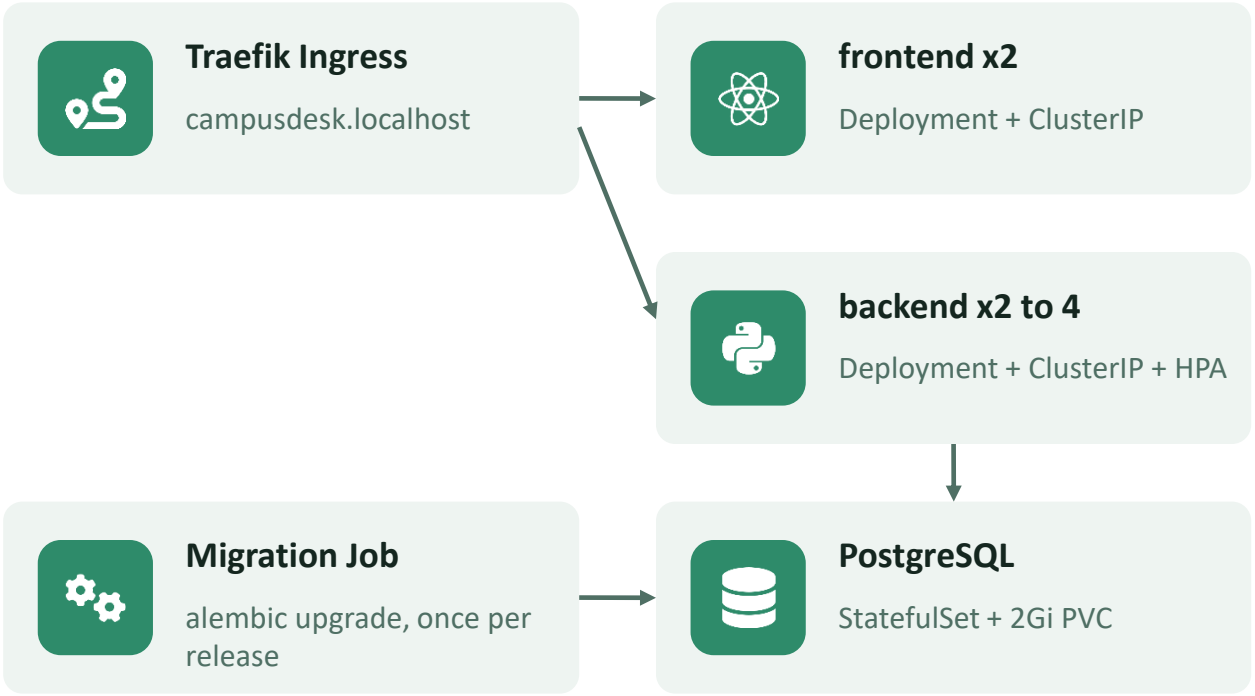
Trivy, exit-code 1



Runtime

non-root, seccomp

Kubernetes + Helm: what the chart creates



CampusDesk | Captured command output: kubernetes

Original recorded output, rendered for readability. Environment: local Windows / Kind.

```
Captured at 2026-10-14T22:06:54.472328+00:00
Command: kubectl --context kind-campusdesk -n campusdesk get pods,svc,ingress,hpa
Exit code: 0

NAME                                READY   STATUS    RESTARTS   AGE
pod/backend-86b4d9b9cb-p5z4q        1/1     Running   0          61s
pod/backend-86b4d9b9cb-zvwkp        1/1     Running   0          76s
pod/frontend-5c9c7cf5-bbm4h         1/1     Running   0          76s
pod/frontend-5c9c7cf5-nbhcz         1/1     Running   0          66s
pod/grafana-57987d6bb8-qjcm5        1/1     Running   0          64m
pod/postgres-0                      1/1     Running   0          64m
pod/prometheus-55556d754b-c68vp     1/1     Running   0          64m

NAME                       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/backend            ClusterIP   10.96.179.39    <none>        8000/TCP         64m
service/backend-metrics    ClusterIP   None            <none>        8000/TCP         64m
service/frontend           ClusterIP   10.96.17.71     <none>        8080/TCP         64m
service/grafana            NodePort    10.96.109.255   <none>        3000:30300/TCP   64m
service/postgres           ClusterIP   None            <none>        5432/TCP         64m
service/prometheus         NodePort    10.96.145.135   <none>        9090:30900/TCP   64m

NAME                                    CLASS     HOSTS                  ADDRESS     PORTS   AGE
ingress.networking.k8s.io/campusdesk    traefik   campusdesk.localhost   127.0.0.1   80      64m

NAME                                                  REFERENCE              TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/backend   Deployment/backend   cpu: 7%/70%   2         4         2          64m
```

`/` to frontend `/api` `/metrics` to backend

Probes: startup + liveness on /health, readiness on /ready. Secrets are generated by Helm and never committed.

GitOps: Git records what is deployed

1 **CI passes every gate**

tests, scans and the kind smoke deploy

2 **CI commits the image tags**

helm/campusdesk/values-release.json, [skip ci]

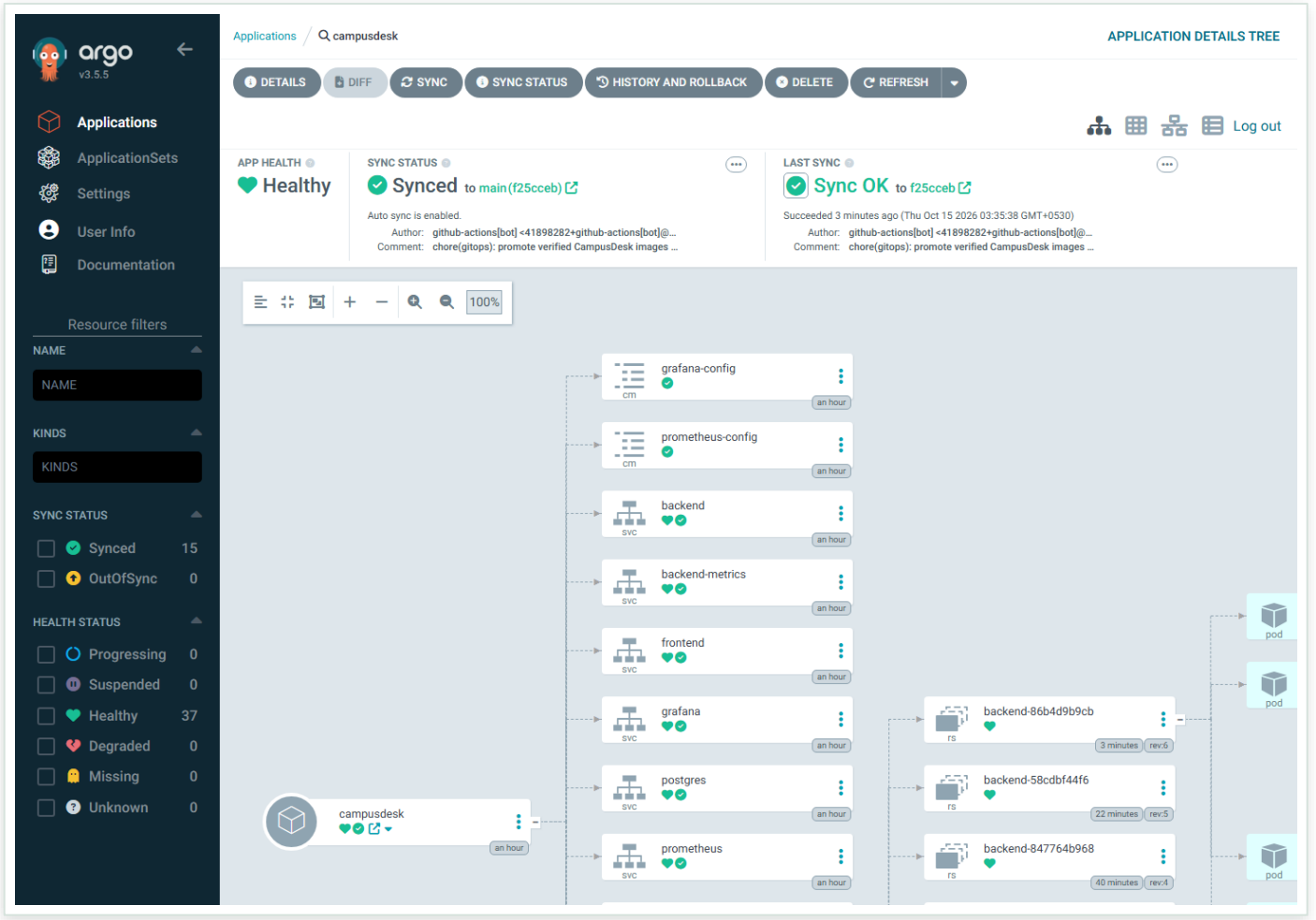
3 **Argo CD sees the commit**

Application tracks main, automated sync

4 **Cluster converges**

self-heal reverts manual drift

Every commit that passes the pipeline is deployed automatically, and nothing else is. Pruning stays off so a bad commit cannot delete the database volume.



Terraform: AWS VPC + EKS as code

VPC 10.42.0.0/16 | ap-south-1

Availability zone 1

Public subnet

10.42.1.0/24 + NAT gateway

Private subnet

10.42.11.0/24

EKS worker
t3.medium

Availability zone 2

Public subnet

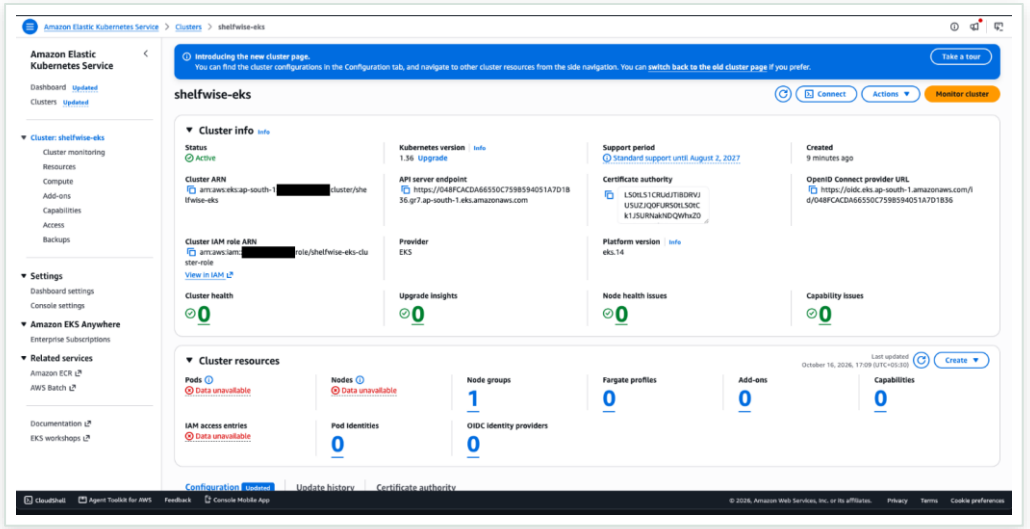
10.42.2.0/24 + load balancers

Private subnet

10.42.12.0/24

EKS worker
t3.medium

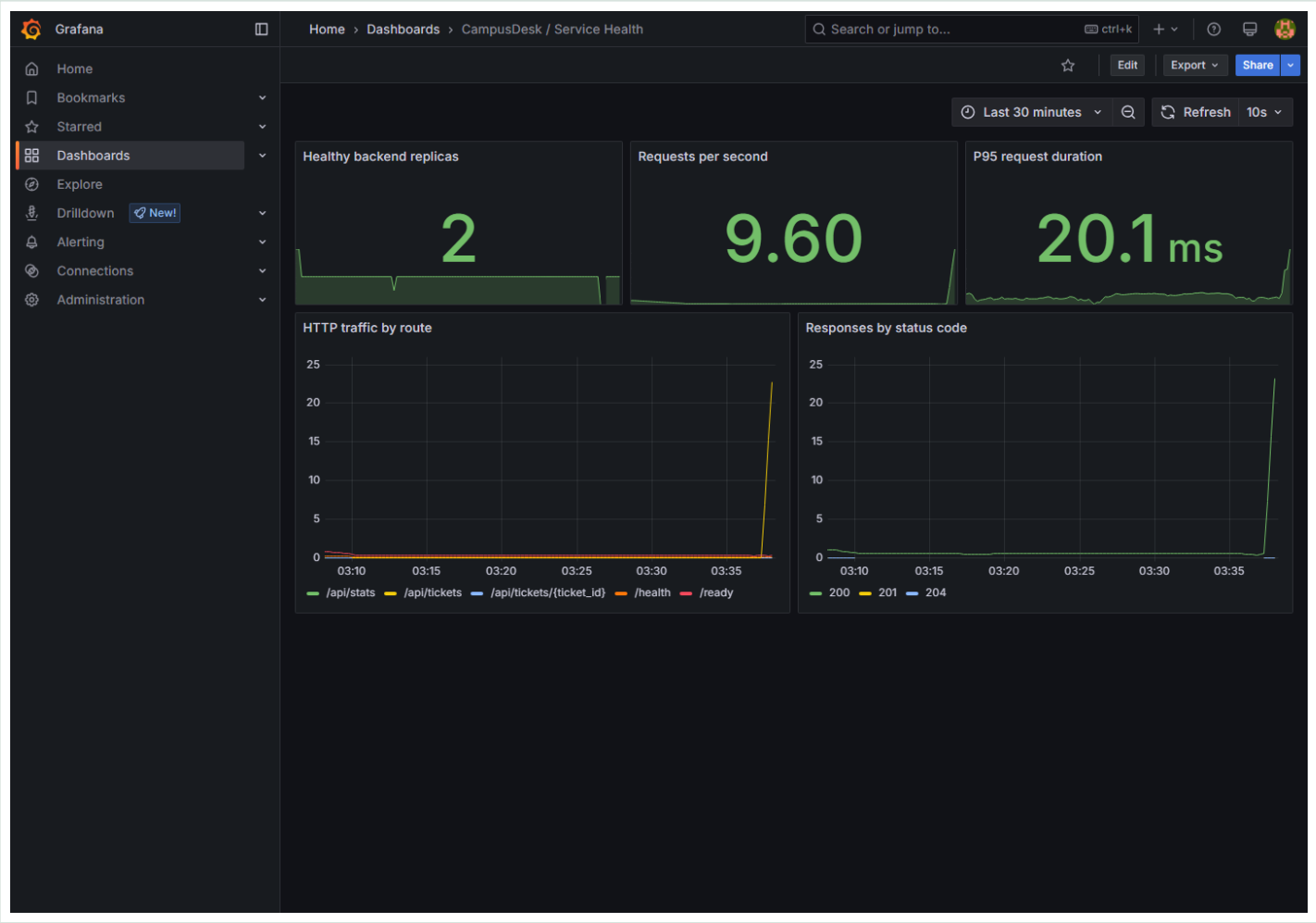
Managed node group: desired 2, scales 1 to 3. EBS CSI role for volumes. API access limited to trusted CIDRs.



`init + validate`	`plan`	`apply`	`destroy`
clean	22 to add	22 added	22 destroyed

Console evidence is from my ShelfWise EKS Terraform run (22 resources, applied and destroyed). The CampusDesk code above is validated in CI on every push.

Observability: Prometheus + Grafana



Custom /metrics

campusdesk_http_requests_total and request duration histograms, labelled by route template

Per-pod scraping

A headless backend-metrics Service lets Prometheus scrape every replica

5 dashboard panels

Healthy replicas, requests/s, P95 latency, traffic by route, status codes

2 alert rules

Backend unavailable 2 min; 5xx above 5% for 5 min

Autoscaling under real load

2 to 4

backend pods (the HPA maximum)

442%

of the 70% CPU target at peak

15,833

successful requests

0

failed requests

CampusDesk | Captured command output: autoscaling

Original recorded output, rendered for readability. Environment: local Windows / Kind.

```
NAME                                        REFERENCE            TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/backend   Deployment/backend   cpu: 442%/70%   2         4         4          29m


NAME                            READY   STATUS    RESTARTS   AGE
pod/backend-847764b968-bshbs    1/1     Running   0          3m6s
pod/backend-847764b968-p257f    1/1     Running   0          39s
pod/backend-847764b968-tjtkw    1/1     Running   0          2m36s
pod/backend-847764b968-z6w69    1/1     Running   0          39s
pod/frontend-78d8685995-fl812   1/1     Running   0          2m55s
pod/frontend-78d8685995-jvcpp   1/1     Running   0          3m6s
pod/grafana-57987d6bb8-qjcm5    1/1     Running   0          29m
pod/postgres-0                  1/1     Running   0          29m
pod/prometheus-55556d754b-c68vp 1/1     Running   0          29m
```

```
python scripts/load.py --seconds 60 --workers 4
```

Troubleshooting labs: symptom, cause, fix



Broken image

SYMPTOM

Pods stuck in ImagePullBackOff

CAUSE

Pod events name a tag that does not exist

FIX

kubectl set image to a real tag, watch the rollout



Broken Service

SYMPTOM

ClusterIP exists, but no endpoints

CAUSE

Selector app=nonexistent-backend matches no pod labels

FIX

kubectl set selector app=backend



Not ready

SYMPTOM

Pod Running but 0/1, no endpoint

CAUSE

Readiness probe on /missing-probe returns 404

FIX

Fix the probe path; the pod becomes Ready

Each lab uses separate demo objects, so the real backend is never broken. helm rollback restores chart config, but not a database migration.

Design decisions and why

/health skips the DB, /ready checks it

A database outage stops traffic instead of restarting healthy pods in a loop

Image tag = full commit SHA

Immutable, traceable deployments: commit A is image A

Publish only after scan + deploy

A vulnerable or broken image is never pushed; one job holds write access

Migrations in a separate Job

One migration per release; backend is ready only once the schema exists

Route templates as metric labels

Ticket IDs as labels would create unbounded time series

Argo CD pruning disabled

A bad commit cannot delete resources such as the database volume

Key learnings and known limits

LEARNED

- A gate only helps if a failure really blocks the release: Trivy stopped a vulnerable image
- Git as the deployment record makes every release traceable to one commit
- Separate liveness and readiness, and run migrations once, not in every replica
- Evidence should show real runs and name the environment it came from

NEXT STEPS

- No authentication or RBAC yet
- Single PostgreSQL instance; production wants RDS + backups
- Plain HTTP Ingress; production adds TLS (cert-manager)
- Local Prometheus/Grafana storage is ephemeral

Thank you github.com/abdulkalamazad1001/scaler-devops-homework